Control Barrier Functions (CBF) — A Beginner-Friendly Explanation

> **Viewing:** Open in VS Code and press **Ctrl+Shift+V** to preview with rendered equations. A PDF version with rendered math is also available in this repo.

The Big Picture — What Is CBF?

Imagine you're driving a car toward a destination. You have a GPS telling you where to go — that's your **nominal controller**. But GPS doesn't know about obstacles. So you also have a **co-pilot** who watches the road and grabs the steering wheel *only* when you're about to crash. The rest of the time, the co-pilot sits quietly and lets you drive.

That co-pilot is the **Control Barrier Function (CBF)**.

More precisely, CBF is a **safety filter**:

1. You design any controller you want to reach the goal (the "nominal controller")
2. At every timestep, CBF checks: "Is this control input safe?"
3. If yes: use it as-is. CBF does nothing.
4. If no: CBF tweaks it *just enough* to avoid collision. The smallest possible change.

The math tool that does step 4 is a **Quadratic Program (QP)** — a small optimisation problem that finds the safe control closest to what you wanted.

How is CBF different from DAF and APF?

	APF	**DAF**	**CBF**
What it does	Pushes robot away from obstacles	Slows robot down near obstacles	Filters your controller for safety
Avoidance is...	Baked into the controller	Baked into the controller	A separate layer on top
Activates when...	Near obstacle (always)	Near obstacle and approaching	Only when your controller would be unsafe
Multiple obstacles	Hard (only nearest)	Hard (only nearest)	Easy — add one constraint per obstacle

	APF	DAF	CBF
Safety proof	None	Yes (Lyapunov)	Yes (forward invariance)

The Logic Flow — How CBF Is Built

Here's the roadmap. Each step builds on the previous one:

1. **Write down the robot model** — "What kind of system are we controlling?"
2. **Define what 'safe' means** — "Where is the robot allowed to be?"
3. **Write the safety rule** — "What must the control input satisfy to stay safe?"
4. **Handle the acceleration problem** — "Our robot controls acceleration, not velocity — we need an extra step"
5. **Build the QP** — "Find the safest control closest to what we want"
6. **Prove it works** — "Once safe, always safe"

Let's walk through each step.

Step 1 — The Robot Model

$$\dot{x} = f(x) + g(x)\,u$$

In plain English: The robot's state x changes over time due to two things:

- $f(x)$: what happens naturally (like gravity, or coasting at current velocity)
- $g(x) \cdot u$: what we command (our control input)

The key requirement is that u enters **linearly** — no u^2 or $\sin(u)$. This is what makes the QP solvable.

For our robot (same as DAF), the state is position and velocity stacked together:

$$x = \begin{bmatrix} p \\ v \end{bmatrix}, \quad f(x) = \begin{bmatrix} v \\ 0 \end{bmatrix}, \quad g(x) = \begin{bmatrix} 0 \\ I \end{bmatrix}$$

This just says: position changes at rate v, and velocity changes at rate u (Newton's law for unit mass).

Symbol	Meaning
x	Full state (position + velocity stacked)
p	Position of the robot

Symbol	Meaning
v	Velocity of the robot
u	Control input — the acceleration we choose
$f(x)$	"Drift" — what happens with no control
$g(x)$	How our control affects the state

Step 2 — Defining "Safe"

We need a mathematical way to say "the robot hasn't crashed." We do this by picking a function $h(x)$ such that:

$$h(x) \geq 0 \quad \iff \quad \text{the robot is safe}$$

The **safe set** is everything where h is non-negative:

$$\mathcal{C} = \{x : h(x) \geq 0\}$$

Think of h as a "safety score":

- $h > 0$: safe (positive score = good)
- $h = 0$: on the edge of safety (score is zero = be careful)
- $h < 0$: collision! (negative score = bad)

Choosing h for obstacle avoidance

For a circular robot (radius R) avoiding a circular obstacle (radius r_o) with centre at p_o:

$$h(p) = \|p - p_o\|^2 - (R + r_o + \epsilon)^2$$

What this says: Take the squared distance between the robot and obstacle centres, and subtract the squared minimum allowed distance. If the result is positive, they're far enough apart.

We use squared distance (instead of plain distance) because it makes the derivatives cleaner — no square roots.

Symbol	Meaning
$h(p)$	Safety score — positive means safe
p	Robot position

Symbol	Meaning
p_o	Obstacle centre
$\|p - p_o\|$	Distance between centres
$R + r_o + \epsilon$	Minimum allowed distance (robot radius + obstacle radius + safety margin)

For a flat wall at position x_w, you'd use: $h(p) = (p_x - x_w)^2 - (R + \epsilon)^2$

Step 3 — The Safety Rule (CBF Condition)

The core idea: **don't let h decrease too fast.** If h can never reach zero, the robot never crashes.

First attempt (first-order systems)

If we could directly control velocity ($\dot{p} = u$), the rule would be:

$$\dot{h} \geq -\alpha(h)$$

where α is a positive function with $\alpha(0) = 0$. Usually just $\alpha(h) = \alpha_0 \cdot h$ (linear).

What this means:

- When h is large (far from obstacle): $\alpha(h)$ is large, so $\dot{h}$ can be quite negative — you're allowed to approach fast.
- When h is small (close to obstacle): $\alpha(h)$ is small, so $\dot{h}$ must be close to zero — slow down!
- When $h = 0$ (at the boundary): $\dot{h} \geq 0$ — you must be moving away or standing still. No crossing allowed.

The function α is like a speed limit that gets stricter the closer you are to the edge.

What is $\dot{h}$?

Using the chain rule and our system model:

$$\dot{h} = \underbrace{\nabla h^\top f(x)}_{L_f h} + \underbrace{\nabla h^\top g(x)}_{L_g h} \cdot u$$

The terms $L_f h$ and $L_g h$ are called **Lie derivatives** — fancy names for "how h changes due to drift" and "how h changes due to control."

So the safety rule becomes:

$$L_f h + L_g h \cdot u + \alpha(h) \geq 0$$

This is a **linear inequality in** u — easy to enforce!

Step 4 — The Acceleration Problem (Relative Degree 2)

Here's the catch: our robot controls **acceleration** ($\dot{v} = u$), not velocity directly. And our barrier function $h(p)$ depends on **position**.

So when we compute $\dot{h}$:

$$\dot{h} = \nabla_p h \cdot v$$

There's no u **in there!** Velocity appears, but not the control. We can't constrain something that doesn't show up.

This is called **relative degree 2**: we need to differentiate h **twice** before u appears:

$$\ddot{h} = 2\|v\|^2 + 2(p - p_o)^\top u$$

Now u is there! So we write the safety rule on $\ddot{h}$ instead:

$$\ddot{h} + \alpha_1 \dot{h} + \alpha_2 h \geq 0$$

This is called the **Exponential CBF (ECBF)** condition.

Think of it like a spring: the equation $\ddot{h} + \alpha_1 \dot{h} + \alpha_2 h \geq 0$ ensures $h(t)$ behaves at least as well as a damped spring that never goes negative. The parameters α_1 and α_2 are the damping and stiffness.

Choosing α_1 **and** α_2**:**

- Both must be positive
- Need $\alpha_1^2 \geq 4\alpha_2$ for guaranteed safety (real poles)
- Larger values = more conservative (intervenes earlier)
- Smaller values = more aggressive (waits longer, gets closer to obstacles)

Expanding for our specific robot + circular obstacle:

Plugging in $h = \|p - p_o\|^2 - r_{\text{safe}}^2$, $\dot{h} = 2(p - p_o)^\top v$, and $\ddot{h} = 2\|v\|^2 + 2(p - p_o)^\top u$:

$$2(p - p_o)^\top u \geq -\Big(2\|v\|^2 + 2\alpha_1 (p - p_o)^\top v + \alpha_2 h(p)\Big)$$

This is one linear inequality in u. That's it — that's the safety constraint.

Piece	Formula	What it means
Left side	$2(p - p_o)^\top u$	"How much are you accelerating away from the obstacle?"

Piece	Formula	What it means
$2\|v\|^2$	Speed term	Faster robot needs more room to brake
$2\alpha_1(p - p_o)^\top v$	Approach rate	Negative when heading toward obstacle — tightens constraint
$\alpha_2 h(p)$	Safety margin	Positive when safe — loosens constraint

---

Step 5 — The QP (Putting It All Together)

Now we have two things:

1. A **nominal controller** that drives toward the goal: $u_{\text{nom}} = -k_1(p - p_d) - k_2 v$
2. A **safety constraint** that prevents collision (from Step 4)

The QP combines them:

$$u^* = \arg\min_u \quad \|u - u_{\text{nom}}\|^2$$

$$\text{subject to:} \quad 2(p - p_o)^\top u + 2\|v\|^2 + 2\alpha_1(p - p_o)^\top v + \alpha_2 h \geq 0$$

In plain English: "Find the acceleration u^* that is as close as possible to what the nominal controller wants, but also satisfies the safety constraint."

What happens in practice:

- If u_{nom} already satisfies the constraint: $u^* = u_{\text{nom}}$. No change. The co-pilot stays quiet.
- If u_{nom} violates the constraint: u^* is the nearest safe point — a projection onto the constraint boundary.

Closed-form solution (single obstacle)

For one obstacle, you don't even need a QP solver. Define:

- $A = 2(p - p_o)^\top$ (a row vector)
- $b = 2\|v\|^2 + 2\alpha_1(p - p_o)^\top v + \alpha_2 h$ (a scalar)

Then:

$$u^* = \begin{cases} u_{\text{nom}}, & \text{if } A \cdot u_{\text{nom}} + b \geq 0 \\ u_{\text{nom}} + \dfrac{-(A \cdot u_{\text{nom}} + b)}{A \cdot A^\top} A^\top, & \text{otherwise} \end{cases}$$

The second case is just a projection — push u_{nom} in the direction $A^\top$ (away from obstacle) by exactly enough to satisfy the constraint.

Multiple obstacles

Just add one constraint per obstacle:

$$\text{s.t.} \quad 2(p - p_{o,i})^\top u + 2\|v\|^2 + 2\alpha_1 (p - p_{o,i})^\top v + \alpha_2 h_i \geq 0, \quad i = 1, \ldots, N_{obs}$$

This is a big advantage over DAF and APF, which only track the nearest obstacle.

Step 6 — Why It's Safe (The Proof Idea)

Claim: If the robot starts safe ($h(x(0)) \geq 0$) and the control always satisfies the CBF condition, then the robot stays safe forever:

$$h(x(0)) \geq 0 \implies h(x(t)) \geq 0 \quad \text{for all } t \geq 0$$

Why: The CBF condition says $\dot{h} \geq -\alpha(h)$. At the boundary ($h = 0$), this becomes $\dot{h} \geq -\alpha(0) = 0$. So h can never decrease past zero — the robot can never cross the safety boundary. This is called **forward invariance** of the safe set.

One requirement: The QP must always have a solution (be "feasible"). For circular obstacle avoidance, this holds as long as the robot starts outside the obstacle, because the control effectiveness $2(p - p_o)^\top$ is nonzero whenever $p \neq p_o$.

DAF vs CBF vs APF — Side by Side

How each handles obstacle avoidance:

DAF bakes everything into one formula:

$$u = \underbrace{-k_1(p - p_d)}_{\text{go to goal}} - \underbrace{k_2 v}_{\text{slow down}} - \underbrace{k_3 \gamma(d)\, \eta\, \eta^\top v}_{\text{brake near obstacles}}$$

CBF keeps them separate:

$$u^* = \underbrace{u_{\text{nom}}}_{\text{go to goal + slow down}} + \underbrace{\lambda \cdot (p - p_o)}_{\text{safety correction (only when needed)}}$$

where $\lambda \geq 0$ is automatically computed by the QP (zero when safe, positive when intervening).

APF adds a repulsive force:

$$u = \underbrace{-k_a(p - p_d)}_{\text{go to goal}} - \underbrace{k_v v}_{\text{slow down}} + \underbrace{k_r F_r(p)}_{\text{push away from obstacles (always)}}$$

Key differences:

Question	APF	DAF	CBF
Does it push when moving away from obstacle?	Yes (wasteful)	No (only brakes when approaching)	No (only intervenes when needed)
Can it handle multiple obstacles?	Nearest only	Nearest only	All at once (one constraint each)
Is there a safety proof?	No	Yes	Yes
What do you tune?	k_a, k_v, k_r	$k_1, k_2, k_3, \epsilon_1, \epsilon_2$	$k_1, k_2, \alpha_1, \alpha_2$
Can you swap the goal controller?	No (coupled)	No (coupled)	Yes (modular)
Gets stuck on flat walls?	Yes	Yes	Yes (but for different reasons)

Implementation Checklist

To code up CBF for a single circular obstacle in 2D:

Inputs at each timestep:

- p — robot position (from sensors)
- v — robot velocity (from sensors)
- p_d — goal position (given)
- p_o, r_o — obstacle centre and radius (from sensors)

Compute:

1. Nominal control: $u_{\text{nom}} = -k_1(p - p_d) - k_2 v$
2. Barrier value: $h = \|p - p_o\|^2 - (R + r_o + \epsilon)^2$
3. Barrier derivative: $\dot{h} = 2(p - p_o)^\top v$
4. Constraint check: $A = 2(p - p_o)^\top$, $b = 2\|v\|^2 + 2\alpha_1 \dot{h}/2 + \alpha_2 h$
    - Wait, more carefully: $b = 2\|v\|^2 + 2\alpha_1 (p - p_o)^\top v + \alpha_2 h$
5. If $A \cdot u_{\text{nom}} + b \geq 0$: use $u^* = u_{\text{nom}}$
6. Else: $u^* = u_{\text{nom}} + \frac{-(A \cdot u_{\text{nom}} + b)}{A \cdot A^\top} A^\top$

Output: Apply u^* to the robot.

For multiple obstacles, use a QP solver (e.g., `scipy.optimize.minimize` or OSQP) with one constraint per obstacle.

Symbol Reference

Symbol	What it means
$x = [p, v]$	Full state (position + velocity)
p	Robot position
v	Robot velocity
u	Control input (acceleration)
p_d	Goal position
p_o	Obstacle centre
R	Robot radius
r_o	Obstacle radius
ϵ	Extra safety margin
$h(x)$	Barrier function (positive = safe)
$\dot{h}$	How fast safety is changing
$\ddot{h}$	How fast $\dot{h}$ is changing
$L_f h$	How h changes due to drift (no control)
$L_g h$	How control affects h
$\alpha(\cdot)$	Function controlling enforcement aggressiveness
α_1, α_2	ECBF tuning parameters (damping, stiffness)
k_1, k_2	Nominal controller gains (goal spring, friction)
u_{nom}	What the nominal controller wants
u^*	What the robot actually does (safe version)

Symbol	What it means
λ	QP multiplier (0 = CBF inactive, >0 = CBF intervening)
$\mathcal{C}$	Safe set — all states where $h \geq 0$
∇h	Gradient of h — direction of increasing safety

Summary

1. **Pick a barrier function** h that is positive when safe, zero at the boundary
2. **Write the safety rule**: $\ddot{h} + \alpha_1 \dot{h} + \alpha_2 h \geq 0$
3. **Solve a QP** at each timestep: closest safe control to what you want
4. **Result**: the robot follows your controller when safe, and deviates minimally when needed

$$\text{Model} \rightarrow \text{Define safety} \rightarrow \text{Safety rule} \rightarrow \text{Handle acceleration} \rightarrow \text{QP} \rightarrow \text{Proof}$$

CBF — Simulation Ideas

Priority	Simulation	What it shows
1	Single circle, CBF-QP	Core implementation works
2	CBF vs DAF vs APF comparison	Why CBF is different
3	Plot $\lambda(t)$ over time	When/how much CBF intervenes
4	Sweep α_1, α_2	How tuning affects conservatism
5	Flat wall (same as DAF sim)	Both methods' stuck behavior
6	Multiple obstacles	QP handles multiple constraints